Module 4: Vault Transfer

Secure Password Manager Project

Rana Elgharabawy | 2026

1. Overview

Module 4 implements secure vault transfer between two users of the Secure Password Manager. It combines Diffie-Hellman (DH) key exchange, AES-GCM symmetric encryption, and ElGamal digital signatures (from Module 3) to allow one user (sender) to export their encrypted credential vault to another user (receiver) without ever exposing plaintext credentials or the master password over the transfer channel.

Key responsibilities of this module:

- Perform DH key exchange to derive a shared session key.
- Re-encrypt the vault under the shared session key for transit.
- Sign the exported vault to guarantee authenticity and integrity.
- Verify the sender's signature on import before accepting credentials.
- Merge imported credentials into the receiver's vault, skipping duplicates.

2. Dependencies

Module / Library	Source	Purpose
module_2	Internal	encrypt_credentials, decrypt_credentials, add_credential
module_3	Internal	sign, verify, hash_vault (ElGamal DSA)
config	Internal	load_params, load_json (DH parameters)
pycryptodome	External	AES-GCM cipher, secure random integer
hashlib	stdlib	SHA-256 key derivation
os, json, base64	stdlib	File I/O, serialisation, encoding

3. Cryptographic Primitives

3.1 Diffie-Hellman Key Exchange

The module reuses the DH parameters (prime q and generator alpha) already stored in each user's public key JSON file. The DH key exchange proceeds as follows:

1. Sender computes shared secret: K = Y_receiver ^ x_sender mod q
2. Receiver computes shared secret: K = Y_sender ^ x_receiver mod q

Both sides arrive at the same K without transmitting private keys.

3.2 Session Key Derivation

The raw DH shared integer K is hashed with SHA-256 to produce a 32-byte (256-bit) AES key:

session_key = SHA-256(str(K).encode())

3.3 AES-GCM Encryption

Credentials are re-encrypted using AES-256-GCM, which provides both confidentiality and authenticated integrity. The wire format (base64-encoded) bundles:

```
Bytes  0-15 : random nonce (16 bytes)
Bytes 16-31 : GCM authentication tag (16 bytes)
Bytes 32+   : ciphertext
```

3.4 ElGamal Digital Signature

Before export, the module signs the re-encrypted vault blob using the sender's ElGamal private key (via module_3.sign). On import, the receiver verifies the signature with the sender's public key before decrypting anything, ensuring the transfer file was not tampered with in transit.

4. Function Reference

select_keys(q, alpha) -> (int, int)

Generates a fresh DH key pair within the group (q, alpha). Picks a random private key x in [2, q-2] and computes public key y = alpha^x mod q.

Parameters:

Parameter	Type	Description
q	int	Safe prime - the DH group order
alpha	int	Generator of the DH group

Returns: Tuple (private_key x, public_key y)

compute_key(otherPubKey, myPrivKey, q) -> int

Computes the DH shared secret: K = otherPubKey ^ myPrivKey mod q.

Parameters:

Parameter	Type	Description
otherPubKey	int	Counterpart's DH public key (y)
myPrivKey	int	Own DH private key (x)
q	int	DH group prime

Returns: Shared secret integer K

hash_key(key) -> bytes

Derives a 32-byte AES key from a DH shared secret integer using SHA-256.

Parameters:

Parameter	Type	Description
key	int	DH shared secret integer K

Returns: 32-byte bytes object suitable for AES-256

AES_encrypt(plaintext: bytes, key: bytes) -> str

Encrypts plaintext with AES-256-GCM using a freshly generated nonce. Returns a base64-encoded string containing nonce || tag || ciphertext.

Parameters:

Parameter	Type	Description
plaintext	bytes	Data to encrypt
key	bytes	32-byte AES session key

Returns: Base64 string: nonce(16) + tag(16) + ciphertext

AES_decrypt(bundle: str, key: bytes) -> bytes

Decrypts a bundle produced by AES_encrypt. Raises ValueError if authentication fails.

Parameters:

Parameter	Type	Description
bundle	str	Base64 string from AES_encrypt
key	bytes	32-byte AES session key

Returns: Decrypted plaintext bytes

Note: Raises ValueError on tag mismatch (tampering detected).

export_vault(sender, receiver, master_password)

High-level export workflow. Loads the sender's vault, verifies its integrity, decrypts it with the master password, re-encrypts it under the DH session key, signs the export, and writes a JSON transfer file to the transfers/ directory.

Parameters:

Parameter	Type	Description
sender	str	Username of the exporting user
receiver	str	Username of the target user
master_password	str	Sender's vault master password

Returns: None - writes transfers/<sender>_to_<receiver>.json

Note: Raises FileNotFoundError if keys/vault are missing; PermissionError if signature invalid.

import_vault(receiver, sender, master_password)

High-level import workflow. Reads the transfer file, verifies the sender's ElGamal signature, decrypts the payload with the DH session key, and merges credentials into the receiver's vault (skipping duplicates). Deletes the transfer file after successful import.

Parameters:

Parameter	Type	Description
receiver	str	Username of the importing user
sender	str	Username of the original exporter
master_password	str	Receiver's own master password

Returns: None - updates receiver's vault and prints import summary

Note: Raises PermissionError if signature verification fails; FileNotFoundError if transfer file missing.

5. Security Design

Confidentiality in transit:

Vault re-encrypted with AES-256-GCM under ephemeral DH session key; master password never leaves the local machine.

Integrity / Authenticity:

ElGamal signature over the exported blob; receiver verifies before decrypting.

Key freshness:

AES_encrypt generates a fresh random nonce per call, preventing nonce reuse.

Tamper detection:

AES-GCM authentication tag rejects modified ciphertext with ValueError.

No master-password exposure:

The master password is used only to decrypt locally; the session key (DH-derived) is used for the export bundle.

Transfer file cleanup:

Transfer file deleted after successful import to limit exposure window.

6. Data Flow Diagram (Export)

```
Sender                                      Receiver
------                                      --------
Load vault (JSON)                           (waiting)
  v
Verify own vault signature  [module_3]
  v
Decrypt vault with master_password  [module_2]
  v
Compute DH session key:                     Compute DH session key:
  K = Y_receiver ^ x_sender mod q             K = Y_sender ^ x_receiver mod q
  session_key = SHA-256(K)                    session_key = SHA-256(K)
  v
Re-encrypt vault with session_key  [AES-GCM]
  v
Sign encrypted_export with x_sender  [module_3]
  v
Write  transfers/sender_to_receiver.json
                                              v
                                            Read transfer file
                                              v
                                            Verify signature  [module_3]
                                              v
                                            Decrypt with session_key  [AES-GCM]
                                              v
                                            Merge into own vault  [module_2]
                                              v
                                            Delete transfer file
```

7. Transfer File Format

Export writes a JSON file at transfers/<sender>_to_<receiver>.json with the following structure:

```
{
  "sender":         "<sender_username>",
  "recipient":      "<receiver_username>",
  "encrypted_vault": "<base64: nonce||tag||ciphertext>",
  "signature": {
    "r": "<ElGamal r component>",
    "s": "<ElGamal s component>"
  }
}
```

8. Test Suite - test_module_4.py

The test file is an integration script that exercises the full export -> import pipeline end-to-end using two test users: alice (sender) and bob (receiver). It is run from the modules/ directory and prints results at each step.

Step 1 - Initialize vaults

module_2.initialize_vault is called for both alice and bob. This creates their DH key pairs in keys/ and empty encrypted vaults in vaults/.

```
module_2.initialize_vault("alice", "alicepass")
module_2.initialize_vault("bob",   "bobpass")
```

Step 2 - Populate alice's vault

Two credentials are added to alice's vault: github.com and netflix.com. Each add_credential call re-signs the vault to maintain integrity.

```
module_2.add_credential("alice", "alicepass", "github.com",  "alice@email.com", "gh_secret")
module_2.add_credential("alice", "alicepass", "netflix.com", "alice@email.com", "nf_secret")
```

Step 3 - Verify alice's vault contents

get_credentials is called to display alice's credentials before export, confirming the vault has the expected entries.

```
for c in module_2.get_credentials("alice", "alicepass"):
    print(c["site"], c["username"], c["password"])
```

Step 4 - Export alice's vault to bob

export_vault performs DH key exchange, re-encrypts alice's vault under the session key, signs it with alice's private key, and writes the transfer file.

```
module_4.export_vault("alice", "bob", "alicepass")
```

Step 5 - Bob imports the vault

import_vault verifies alice's signature, decrypts using the DH session key, and merges all credentials into bob's vault. The transfer file is deleted on success.

```
module_4.import_vault("bob", "alice", "bobpass")
```

Step 6 - Verify bob's vault contents

get_credentials is called on bob's vault to confirm that github.com and netflix.com credentials were imported correctly.

```
for c in module_2.get_credentials("bob", "bobpass"):
    print(c["site"], c["username"], c["password"])
```

9. Expected Test Output

```
=== Step 1: Initialize vaults ===
Vault initialized for 'alice'
Vault initialized for 'bob'

=== Step 2: Add credentials to alice's vault ===
```

```
Credential for 'github.com' added successfully.
Credential for 'netflix.com' added successfully.

=== Step 3: alice's vault before export ===
  github.com  | alice@email.com | gh_secret
  netflix.com | alice@email.com | nf_secret

=== Step 4: Export alice's vault to bob ===
Vault exported successfully to 'bob'.
Transfer file: transfers/alice_to_bob.json

=== Step 5: Bob imports the vault ===
Import complete: 2 credential(s) imported, 0 skipped (already exist).
Transfer file cleaned up.

=== Step 6: Bob's vault after import ===
  github.com  | alice@email.com | gh_secret
  netflix.com | alice@email.com | nf_secret
```

10. Error Handling

Condition	Exception	Where Raised
Sender keys not found	FileNotFoundError	export_vault
Receiver keys not found	FileNotFoundError	export_vault
Sender vault not found	FileNotFoundError	export_vault
Vault signature missing/invalid	PermissionError	export_vault
Transfer file not found	FileNotFoundError	import_vault
Transfer signature invalid	PermissionError	import_vault
Receiver keys not found	FileNotFoundError	import_vault
Receiver vault not found	FileNotFoundError	import_vault
AES tag mismatch (tampered)	ValueError	AES_decrypt